

# 10-bit SAR ADC Design in Sky130 – Project Report

## Contents

|          |                                        |          |
|----------|----------------------------------------|----------|
| <b>1</b> | <b>Introduction and Specifications</b> | <b>2</b> |
| <b>2</b> | <b>Architecture Overview</b>           | <b>3</b> |
| <b>3</b> | <b>Block Design and Sizing</b>         | <b>4</b> |
| 3.1      | Python Behavioral Model . . . . .      | 4        |
| 3.2      | CDAC . . . . .                         | 6        |
| 3.3      | Bootstrapped Switch . . . . .          | 7        |
| 3.4      | StrongARM Comparator . . . . .         | 9        |

# 1 Introduction and Specifications

The successive-approximation (SAR) ADC is the most commonly deployed topology of analog-to-digital converter. Owing to its low power consumption, low latency and compact form factor, it has proven supremely versatile, with notable usage in IoT devices, medical equipment and microcontrollers. This document describes an original 10-bit SAR ADC designed and verified in the SkyWater Sky130 open-source 130nm process — a CMOS PDK developed by SkyWater Technology and Google, providing full device model access without NDA — targeting the specifications in Table 1. The design covers the complete front-end flow: behavioral modeling from first principles, transistor-level schematic capture in xschem, ngspice simulation against Sky130 device models, and Verilog RTL synthesized to Sky130 standard cells via Yosys. Physical layout is out of scope; the project targets schematic-level verification.

Table 1: Design Specifications

| Parameter                    | Value                          |
|------------------------------|--------------------------------|
| Resolution                   | 10 bits                        |
| Sample Rate                  | $1 \frac{\text{MS}}{\text{s}}$ |
| ENOB                         | $\geq 9$ bits                  |
| Supply Voltage, $V_{DD}$     | 1.8 V                          |
| Reference Voltage, $V_{ref}$ | 1.8 V                          |
| Input Range                  | 0 – 1.8 V                      |
| Process                      | SkyWater Sky130, 130nm         |

The design followed a bottom-up flow, progressing sequentially through each of the following stages:

1. First-principles sizing and yield analysis via Python behavioral model
2. Block-level schematic capture in xschem and isolation verification in ngspice
3. Verilog RTL synthesis to Sky130 standard cells via Yosys
4. Full-ADC SPICE integration and static performance characterisation

This document reports the design decisions, block-level verification, integration debugging and static characterisation of the 10-bit SAR ADC. Ultimately, the design achieves a static ENOB of 7.27 bits, falling short of its 9-bit target, attributable to a characterised top-plate feedthrough mechanism, detailed in Section ??.

## 2 Architecture Overview

The design is comprised of four major blocks: the bootstrapped switch, CDAC binary-weighted array, StrongARM comparator and SAR logic block. During the sampling phase, the bootstrapped switch connects  $V_{in}$  to all bottom plates of the CDAC array simultaneously, while the top plate is reset to  $V_{cm} = V_{ref}/2$ . This establishes a charge on the top plate node of  $Q = (V_{ref}/2 - V_{in}) \cdot C_{total}$ . At the start of conversion, the bootstrapped switch is disconnected, leaving the top plate floating with its charge preserved while the bottom plate of each node is shorted to ground. The SAR logic then performs a binary search from the MSB downward: each trial bit steers its corresponding bottom plate to  $V_{ref}$ , shifting  $V_{DAC}$  upward by an amount proportional to that capacitor's weight. The StrongARM comparator evaluates whether  $V_{DAC}$  has crossed  $V_{ref}/2$ , equivalent to testing whether  $V_{in}$  exceeds the analog value of the current trial code. The SAR logic retains or clears each bit accordingly:

1. Assert trial code 1000000000; bottom plate of  $512C$  switches to  $V_{ref}$ , shifting  $V_{DAC}$  upwards.
2. If  $V_{DAC} < V_{ref}/2$  (equivalently,  $V_{in} > V_{ref}/2$ ), retain bit 9; otherwise clear it and return that bottom plate to ground
3. Assert the next trial bit and repeat in descending order until all 10 bits are resolved

This process is summarized by the block diagram in figure 1.

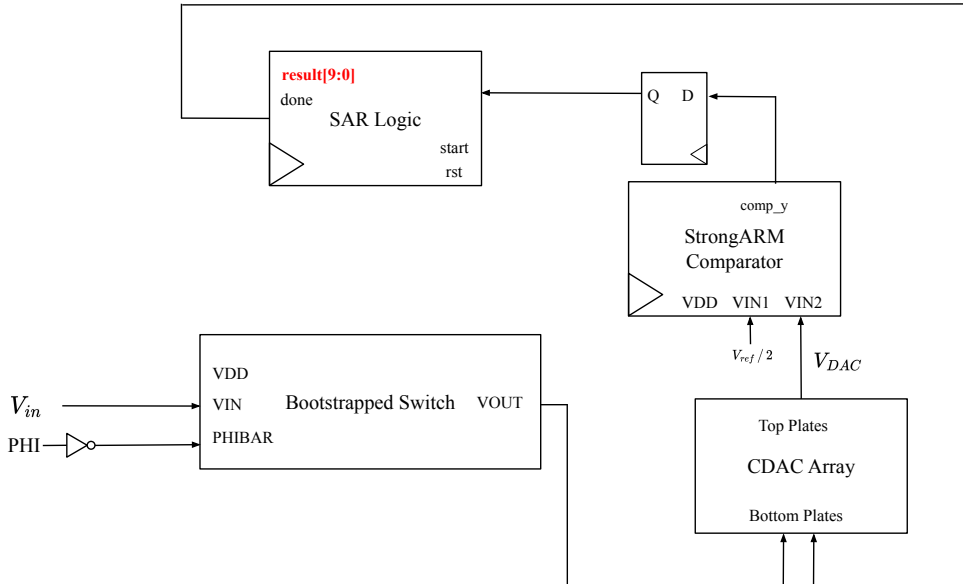


Figure 1: Block diagram of SAR ADC

### 3 Block Design and Sizing

#### 3.1 Python Behavioral Model

The minimum valid unit capacitance,  $C_{unit}$ , that can be used for any capacitor in the CDAC array is governed by the thermal noise floor in equation 1.

$$C_{min} = \frac{kT}{V_{LSB}^2} \quad (1)$$

with  $V_{LSB} = \frac{V_{ref}}{2^N}$ , where  $N$  is the resolution of the instrument. Substituting  $k = 1.38 \times 10^{-23} \text{ J} \cdot \text{K}$ ,  $T = 300 \text{ K}$ ,  $V_{ref} = 1.8 \text{ V}$  and  $N = 10$ ,  $C_{min}$  is found as  $5.36 \text{ fF}$ .

In order to determine a suitable unit capacitance, a Monte Carlo mismatch analysis was conducted. In each trial,  $C_{unit}$  was swept from  $5 \text{ fF}$  to  $54 \text{ fF}$  and a binary-weighted capacitor array,  $[512, 256, 128, \dots, 2, 1, 1] \times C_{unit}$  was generated, representing the CDAC array. Each capacitor in this array was perturbed according to  $\sigma_{C/C} / \sqrt{n_{cells}}$ , where the mismatch  $\sigma_{C/C}$  is  $2\%$ , and  $n_{cells} = C_{unit} / C_{min}$ . Using this representation of the CDAC array, the SAR conversion algorithm was executed on  $2^{16} = 65\,536$  input voltages, 64 per binary code. For each trial, the DNL and INL were computed and the maximum INL was recorded. This process was repeated for 10 000 trials.

The yield is computed as the fraction of trials where the maximum INL is less than  $0.5 \text{ LSB}$ . As can be seen in figure 2, the yield surpasses its target of  $99\%$  at approximately  $24 \text{ fF}$ . Therefore, to allow for some margin, a target value of  $C_{unit} = 25 \text{ fF}$  was selected, giving a total CDAC capacitance of  $1024 \times 25 \text{ fF} = 25.6 \text{ pF}$ .

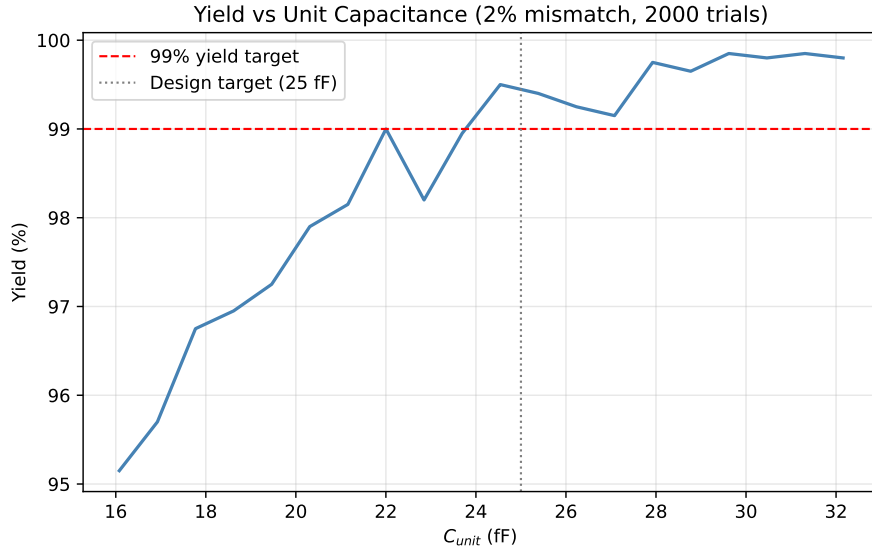


Figure 2: Yield vs.  $C_{unit}$  sweep across 10 000 Monte Carlo trials at  $\sigma_{C/C} = 2\%$ .

To validate the selected  $C_{unit} = 25 \text{ fF}$ , a further 10 000-trial Monte Carlo simulation was run at this fixed value. Figure 3 shows the resulting distribution of maximum INL

across all trials. The distribution is centred near 0.2 LSB, with a tail that barely reaches the 0.5 LSB threshold, confirming a yield of 99.2%.

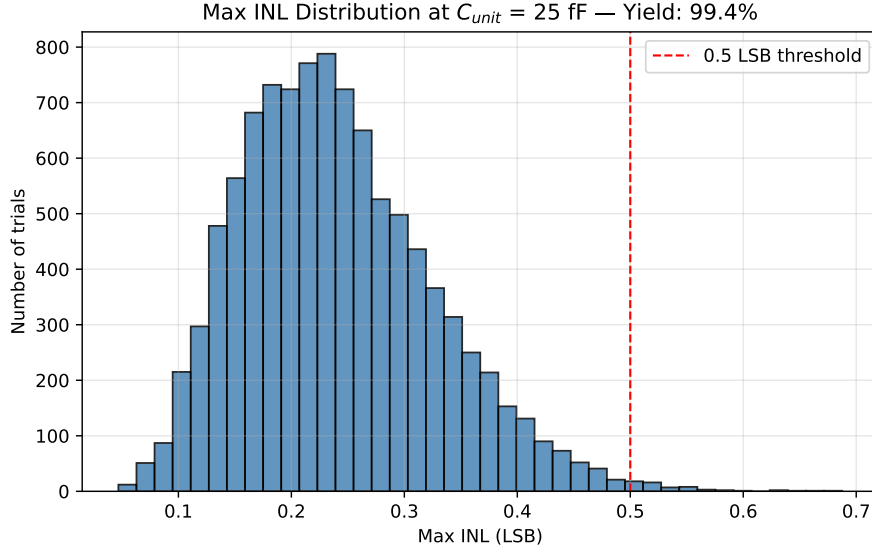


Figure 3: Distribution of maximum INL at  $C_{unit} = 25$  fF, 10 000 Monte Carlo trials,  $\sigma_{C/C} = 2\%$ . Yield: 99.2%.

Figure 4 shows the DNL and INL of a representative single trial at  $C_{unit} = 25$  fF. The DNL remains well within  $\pm 0.5$  LSB across all 1024 output codes, with no missing codes. The INL exhibits a random-walk character, as expected from independent capacitor mismatch errors accumulating across the code range, and stays within  $\pm 0.5$  LSB throughout.

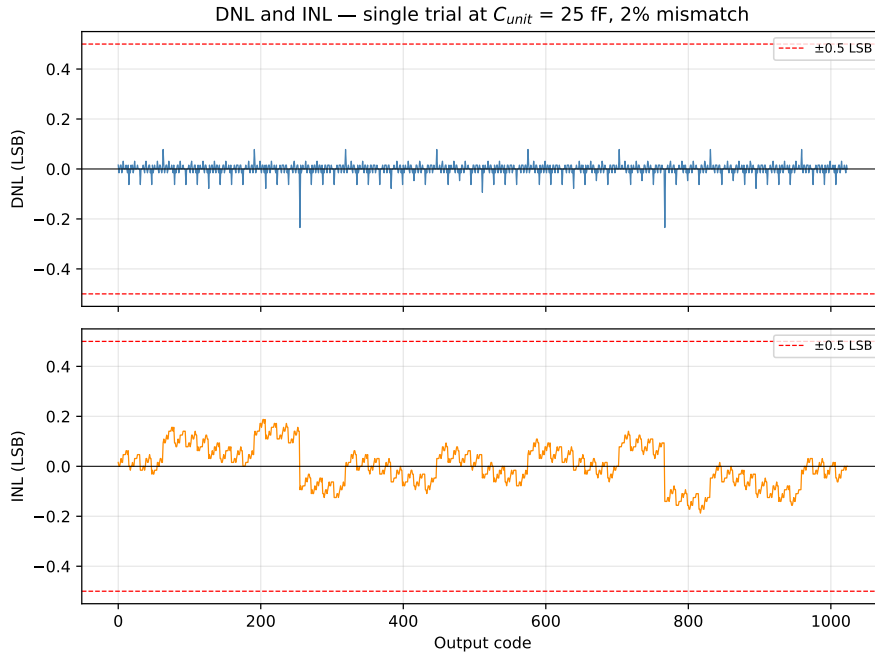


Figure 4: DNL and INL for a single representative trial at  $C_{unit} = 25$  fF,  $\sigma_{C/C} = 2\%$ .

### 3.2 CDAC

The CDAC is a binary-weighted array of capacitors,  $[512, 256, 128, \dots, 2, 1, 1] \times C_{unit}$ . Including the termination capacitor of capacitance  $C_{unit}$ , it spans 11 capacitors with a total capacitance of  $1024 \cdot C_{unit} = 25.6 \text{ pF}$ . The top plate of each capacitor is tied to the common node  $V_{DAC}$ , while each capacitor has a unique bottom plate node. During operation, the CDAC undergoes two stages:

1. Sampling: the bottom plate of each capacitor is driven to  $V_{in}$ , while  $V_{DAC}$  is driven to  $V_{cm} = V_{ref} / 2$ . This locks in a charge of  $Q = 1024 \cdot C_{unit} \cdot (V_{ref} / 2 - V_{in})$  onto the CDAC.
2. Conversion:  $V_{DAC}$  is disconnected from its source and becomes floating. The SAR logic block drives the bottom plate of each capacitor to either  $V_{ref}$  or ground, depending on whether the corresponding bit in the code under trial is high or low. For example, if the code under trial is 1010000000, then the bottom plates of  $C_9 = 512C_{unit}$  and  $C_7 = 128C_{unit}$  are sent to  $V_{ref}$ , with the remaining bottom plates being shorted to ground. At the end of conversion,  $V_{DAC} = \frac{W}{1024} V_{ref} + V_{ref} / 2 - V_{in}$ , where  $W$  is the sum of the weights of each capacitor whose bottom plate is at  $V = V_{ref}$ . Note that during conversion, the bottom plate of the termination capacitor is always shorted to ground.

For each analog voltage under test, sampling occurs once at the start of the procedure, while conversion happens once per bit, progressing from the MSB to the LSB. The transistor-level logic that controls the CDAC's behavior is shown in figure 5.

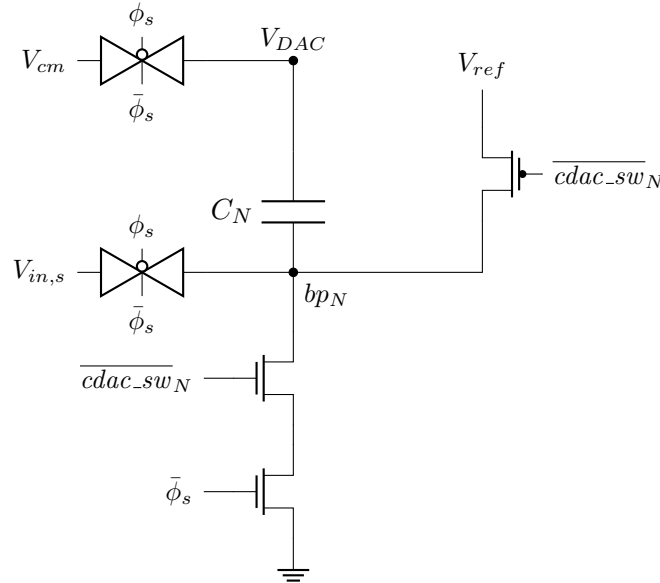


Figure 5: Transistor-level design of one CDAC cell

The diagram in figure 5 applies to all capacitors in the CDAC except for the termination capacitor. Note that for compactness, the transmission gate responsible for charging

$V_{DAC}$  to  $V_{cm}$  during sampling is included in figure 5; however, because  $V_{DAC}$  is shared by all capacitors, the entire CDAC only uses one such transmission gate rather than one per cell. The signal  $cdac\_sw_N$  represents the high/low value of  $N^{th}$  trial bit in the code under test. The capacitor  $C_N$  is responsible for the voltage contribution of this bit to  $V_{DAC}$ , and has a capacitance value of  $C_N = 2^N \cdot C_{unit}$ .

During sampling,  $\phi_s$  is high, meaning that both transmission gates are active and  $V_{DAC}$  gets sent to  $V_{cm}$  while each  $V_{bp_N}$  is sent to  $V_{in}$ . Thereafter,  $\phi_s$  turns off and both transmission gates are switched off, leaving  $V_{DAC}$  floating and initiating the conversion stage. If  $cdac\_sw_N$  is high, then its complement  $\overline{cdac\_sw_N}$  is low and the PMOS of the pull-up network turns on, shorting  $bp_N$  to  $V_{ref}$ . Otherwise, the top NMOS in the pull-down network is activated and  $bp_N$  is routed to ground. The additional NMOS gated by  $\bar{\phi}_s$  in the pull-down network is introduced due to contention; if it were not there, then the pull-down network would provide a path to ground during sampling while  $cdac\_sw_N$  is low. Since  $\phi_s$  is guaranteed to be high during sampling and low during conversion, this NMOS ensures that the pull-down network is never active during sampling but can still be active during conversion.

Note that due to its repetitive nature, no schematic for the CDAC array was designed by hand, with `cdac.spice` instead being written directly.

### 3.3 Bootstrapped Switch

Bootstrapping is necessary in order to avoid bandwidth modulation and harmonic distortion when charging the bottom plates of the CDAC up to  $V_{in}$ . When an NMOS is used to charge up the CDAC, the channel resistance  $R_{on}$  of the FET and the capacitance of the CDAC create a low-pass filter with cutoff frequency given by equation 2.

$$f_n = \frac{1}{2\pi R_{on} C_{DAC}} \quad (2)$$

The channel resistance  $R_{on}$  of an NMOS is given by equation 3.

$$R_{on} = \frac{1}{\mu_n C_{ox} \frac{W}{L} (V_{GS} - V_{th,n})} \quad (3)$$

If the FET is gated by  $V_{DD}$ , then  $V_{GS} = V_{DD} - V_{in}$ , meaning that if  $V_{in}$  changes, the bandwidth of the low-pass filter varies, attenuating signals inconsistently. This unevenness results in harmonic distortion, lowering the effective resolution of the instrument. The bootstrapped switch combats this by lifting the gate voltage of the FET to  $V_{DD} + V_{in}$ , therefore fixing  $V_{GS}$  at  $V_{DD}$ , thus fixing  $R_{on}$  and the corresponding cutoff frequency of the transfer function.

The topology of the bootstrapped switch is shown in figure 6. It operates in two stages.

1. When  $\phi_s$  is low, the PMOS  $Q_2$  is off while the NMOS  $Q_5$  is on, therefore wiring the node X to ground. As such, the NMOS's  $Q_4$  and  $Q_0$  are off, while the PMOS  $Q_1$  and the NMOS  $Q_3$  are on. Therefore, there is a direct pathway from  $V_{DD}$  to ground through  $C_{boot}$ , which charges up to  $V_{DD}$ .
2. When  $\phi_s$  turns on, transistors  $Q_3$  and  $Q_5$  turn off while transistor  $Q_2$  turns on, allowing  $V_X = V_{DD} + V_{in}$ . This causes transistor  $Q_1$  to turn off and transistors  $Q_4$  and  $Q_0$  to turn on, thus providing a pathway for  $V_{in}$  to flow into  $V_{in,s}$  with the gate of  $Q_0$  fixed at  $V_X$ .

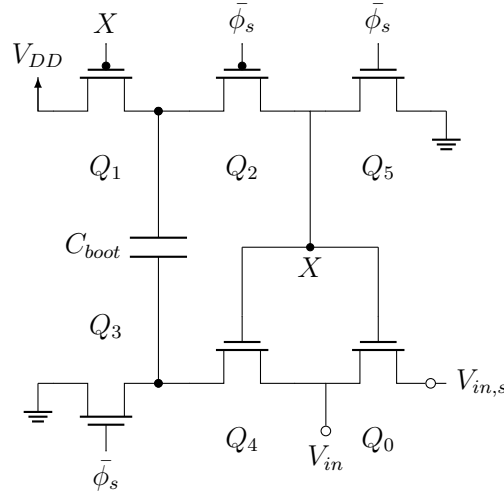


Figure 6: Bootstrapped sampling switch.  $Q_0$  is the main sampling device;  $C_{boot}$  is precharged to  $V_{DD}$  during  $\bar{\phi}_s$  and bootstraps the gate of  $Q_0$  to  $V_{in} + V_{DD}$  during  $\phi_s$ .

Together with the CDAC, the bootstrapped switch enforces the three-phase behavior of the signal  $\phi_s$ . During phase 1,  $\phi_s$  must be low so as to allow  $C_{boot}$  to charge. Then,  $\phi_s$  must become high to turn the switch on and charge up the CDAC. Finally,  $\phi_s$  permanently turns off, disconnecting the CDAC from the bootstrapped switch which has accomplished its purpose by fixing charge onto the CDAC.

In totality, this means that  $\phi_s$  must be a pulse which must start at a delay. The length of this pulse determines the sizing of the transistors and the value of  $C_{boot}$ . This length, in turn, is determined by the sample rate design target, 1 megasample per second.

A rate of 1 megasample per second implies a period of  $1\mu s$ . Using a typical 70-30 split, with 30% of the period allotted to sampling, the width of the pulse is 300 ns. During the width of this pulse, the CDAC must be charged up enough to render an error due to rounding impossible. Practically, this means that the error of the CDAC's voltage must be less than half of an LSB. The error of the CDAC's voltage, defined as the difference between  $V_{in}$  and  $V_{DAC}$ , is given by equation 4.

$$\text{Error} = V_{in} \cdot e^{-t/R_{on}C_{DAC}} \quad (4)$$



Requiring this error to be less than the value of half of one LSB,  $\text{Error} < \frac{V_{ref}}{2^{N+1}}$ , and assuming a worst-case scenario of  $V_{in} = V_{ref}$ , an expression for  $R_{on}$  can be derived.

$$R_{on} \leq \frac{t}{C_{DAC} \cdot 2^{N+1} \cdot \ln 2} \quad (5)$$

Substituting  $t = 300 \text{ ns}$ ,  $C_{DAC} = 25.6 \text{ pF}$  and  $N = 10$ ,  $R_{on}$  is found as  $1.54 \text{ k}\Omega$ . Applying the channel resistance equation of equation 3, with  $V_{GS} = V_{DD} = 1.8 \text{ V}$  due to bootstrapping,  $\mu_n C_{ox} = 270 \frac{\mu\text{A}}{\text{V}^2}$  as set by Sky130, and  $l = 150 \text{ nm}$  (the minimum allowable channel length in Sky130), the minimum value for  $w$  is found as  $273.8 \text{ nm}$ . Allowing for an ample margin, the transistor  $Q_0$  was chosen to have a width of  $1 \mu\text{m}$ . Auxiliary transistors  $Q_1$  and  $Q_{3-5}$  were sized to match  $Q_0$ 's width layout for convenience, while  $Q_2$  was sized conservatively with a width double that of  $Q_0$ . All transistors have an identical length of  $150 \text{ nm}$ .

Using these design parameters, a value of  $C_{boot}$  was selected that sufficiently minimizes droop. The voltage droop on  $C_{boot}$  due to charge sharing with the parasitic gate capacitance of  $Q_0$ ,  $C_{gate}$ , is given by equation 6.

$$\frac{\Delta V}{V_{DD}} = \frac{C_{gate}}{C_{gate} + C_{boot}} \quad (6)$$

Setting this quantity to be strictly less than 0.01,

$$\begin{aligned} \frac{C_{gate}}{C_{gate} + C_{boot}} &< 0.01 \\ 100 \cdot C_{gate} &< C_{boot} + C_{gate} \\ \boxed{C_{boot} > 99 \cdot C_{gate}} \end{aligned} \quad (7)$$

The gate capacitance,  $C_{gate}$ , is given by  $C_{gate} = wl \cdot C_{ox}$ . Substituting  $w = 1 \mu\text{m}$ ,  $l = 150 \text{ nm}$  and  $C_{ox} = 17 \frac{\text{fF}}{\mu\text{m}^2}$  as set by Sky130,  $C_{boot} > 252.45 \text{ fF}$ . To significantly exceed this mark,  $C_{boot}$  was chosen as  $500 \text{ fF}$ . The final design choices are summarized in table 2.

Table 2: Sizing Choices

| Parameter               | Value            |
|-------------------------|------------------|
| Width of $Q_0$          | $1 \mu\text{m}$  |
| Channel length of $Q_0$ | $150 \text{ nm}$ |
| $C_{boot}$              | $500 \text{ fF}$ |

### 3.4 StrongARM Comparator

The StrongARM comparator is the block which performs the bitwise retain or reject decision making for each trial code. Its topology is shown in figure 7.



$$\begin{aligned}
V_{in1} &> V_{in2} \\
V_{ref} / 2 &> V_{DAC} \\
V_{ref} / 2 &> \frac{W}{1024} V_{ref} + V_{ref} / 2 - V_{in} \\
0 &> \frac{W}{1024} V_{ref} - V_{in} \\
\boxed{V_{in} &> \frac{W}{1024} V_{ref}} \tag{8}
\end{aligned}$$

Therefore, when Y is high, the trial bit is asserted according to the SAR logic of section 2.